

Practical-Week3

1. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Initial Process:\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("Child Process - Running\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());

        sleep(5);

        printf("\nChild Process - After Sleep\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
    }
}
```

```

manikanta@Hemu:~$ nano process_state.c
manikanta@Hemu:~$ gcc process_state.c -o process_state
manikanta@Hemu:~$ ./process_state
Initial Process:
PID = 1328
PPID = 1263

Parent Process - Running
PID = 1328
PPID = 1263
Child Process - Running
PID = 1329
PPID = 1328
Parent Process - Waiting for Child
Child Process - After Sleep
PID = 1329
PPID = 1328
Parent Process - Child Completed

```

```

manikanta@Hemu:~$ ps -o pid,ppid,state,cmd
  PID    PPID S  CMD
  1263    1257 S  -bash
  1344    1263 R  ps -o pid,ppid,state,cmd

```

Report: I ran process_state.c, which forked a child (PID 1329) from the parent (PID 1328), with correct PPID linkage. The parent waited while the child ran, then completed. ps -o pid,ppid,state,cmd confirmed correct PID/PPID relationships and process states (S for sleeping, R for running). This confirms fork() correctly creates a child process with proper parent-child linkage.

2. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```

manikanta@Hemu:~$ sleep 30 &
[1] 1503
manikanta@Hemu:~$ ps -p 3521 -o pid,ppid,state,cmd
  PID    PPID S  CMD

```

```
manikanta@Hemu:~$ sleep 60 &  
[1] 1536
```

```
manikanta@Hemu:~$ cat /proc/1536/status | grep -E "Name|State|Pid|P  
Pid":    sleep  
State:   S (sleeping)  
Pid:     1536  
PPid:    1458  
TracerPid:      0
```

```
manikanta@Hemu:~$ ps -p 1536 -o pid,ppid,state,cmd  
  PID    PPID  S   CMD  
 1536    1458  S  sleep 60
```

```
manikanta@Hemu:~$ top  
top - 13:31:33 up 55 min,  1 user,  load average: 0.00, 0.01, 0.00  
Tasks:  27 total,   1 running, 26 sleeping,   0 stopped,   0 zombie  
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st  
MiB Mem :  7782.6 total,  6918.0 free,   554.4 used,   463.9 buff/cache  
MiB Swap: 2048.0 total,  2048.0 free,    0.0 used.  7228.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
92	root	20	0	35368	12252	9164	S	0.3	0.2	0:01.42	systemd
1	root	20	0	24336	15488	11628	S	0.0	0.2	0:01.00	init
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.00	kthreadd
7	root	20	0	3180	2040	1940	S	0.0	0.0	0:00.00	rcu_sched
47	root	19	-1	50400	16852	15548	S	0.0	0.2	0:00.32	systemd-journal
83	systemd+	20	0	22416	14476	11980	S	0.0	0.2	0:00.17	systemd-udev
111	root	20	0	2888	1952	1820	S	0.0	0.0	0:00.03	systemd-timesyn
112	root	20	0	4472	3148	2900	S	0.0	0.0	0:00.01	systemd-network
113	message+	20	0	8796	5524	4832	S	0.0	0.1	0:00.06	dbus-daemon
161	root	20	0	44096	29704	14564	S	0.0	0.4	0:00.20	systemd-logind
...											
1458	manikanta+	20	0	6784	6068	3868	S	0.0	0.1	0:00.06	bash
1536	manikanta+	20	0	3188	1112	980	S	0.0	0.0	0:00.00	sleep
1580	manikanta+	20	0	8172	6056	3948	R	0.0	0.1	0:00.04	ps
1585	root	20	0	35372	6432	3308	S	0.0	0.1	0:00.00	sleep
1586	root	20	0	35372	6432	3308	S	0.0	0.1	0:00.00	sleep

Report: I ran `sleep 60 &` as a background process and inspected it using `/proc/PID/status`, `ps`, and `top`. `/proc/1536/status` showed State: S (sleeping) with correct PID (1536) and PPID (1458) linkage, and `ps -p 1536` confirmed the same sleeping state (S) with the `sleep 60` command. `top` showed the overall system process list, with most processes in state S (sleeping) and one in state R (running). This confirms that background processes can be tracked consistently across `/proc`, `ps`, and `top`, all agreeing on process state and parent-child relationships.